Natural Language Processing

UniVR - Dipartimento di Informatica

Tommaso Toffali

1° Semestre 2026/2027

Contents

I String Similarity and Edit Distance 2

1 Minimum Edit Distance 2

1.1 How similar are two strings? . . . 2

1.1.1 Spell Correction . . . 2

1.1.2 Computational Biology . . . 2

1.1.3 Other Applications . . . 3

1.2 Edit Distance . . . 3

1.2.1 Edit Operations . . . 3

1.2.2 Key Concept . . . 3

1.2.3 Example: Alignment of Two Strings . . . 4

1.2.4 Cost variants: Standard vs Levenshtein . . . 4

1.3 Alignment in Computational Biology . . . 5

1.3.1 The alignment problem . . . 5

1.3.2 Alignment Objective . . . 5

Part I
String Similarity and Edit Distance

1 Minimum Edit Distance

We are going to deal with this main driving point: the definition of Minimum Edit Distance.

The question: are this 2 texts the same?

When is the case when 2 texts are the same? Of course when every single character is the same. What if we would like to understand if 2 texts are pretty close (not the same)?

Single characters in the text could be different in position.

1.1 How similar are two strings?

The fundamental question in edit distance is: how can we measure the similarity between two strings? This problem appears in many different applications across various domains of computer science and computational linguistics.

1.1.1 Spell Correction

In spell correction systems, we need to find the closest valid word to a misspelled input. For example, if the user typed "graffe", we need to determine which word in our dictionary is most similar.

Possible candidates:

- graf
- graft
- grail
- giraffe

By computing the edit distance between "graffe" and each candidate, we can identify that "giraffe" is likely the intended word (requiring only one deletion).

1.1.2 Computational Biology

In bioinformatics, sequence alignment is crucial for comparing DNA, RNA, or protein sequences. By aligning sequences, we can identify regions of similarity that may indicate functional, structural, or evolutionary relationships.

Example: Align two sequences of nucleotides:

```
AGGCTATCACCTGACCTCCAGGCCGATGCCC
```

```
TAGCTATCACGACCGCGGTCGATTTGCCCGAC
```

Resulting alignment (dashes represent insertions/deletions):

```
-AGGCTATCACCTGACCTCCAGGCCGA-TGCCC--
TAG-CTATCAC-GACCGC-GGTCGATTTGCCCGAC
```

The alignment reveals matching regions (shown in the same positions) and differences between the sequences.

1.1.3 Other Applications

String similarity and edit distance algorithms are fundamental tools in many NLP tasks:

- **Machine Translation**: Comparing source and target language phrases, finding similar translations in translation memories.
- **Information Extraction**: Matching entity names with variations (e.g., "IBM" vs "I.B.M." vs "International Business Machines")
- **Speech Recognition**: Correcting recognition errors by finding the closest valid word or phrase to the acoustic model output

1.2 Edit Distance

The **minimum edit distance** between two strings is defined as the minimum number of editing operations needed to transform one string into the other

1.2.1 Edit Operations

Definizione 1.1 (Three Basic Edit Operations)

There are three basic editing operations:

- **Insertion**: Add a character at any position in the string
  - Example: cat $\rightarrow$ cart (insert 'r')
- **Deletion**: Remove a character from any position in the string
  - Example: cart $\rightarrow$ cat (delete 'r')
- **Substitution**: Replace one character with another
  - Example: cat $\rightarrow$ bat (substitute 'c' with 'b')

1.2.2 Key Concept

The edit distance measures the *minimum* number of these operations required to transform one string into another. This metric provides a quantitative measure of string similarity: the smaller the edit distance, the more similar the strings are.

Important note: Each operation has a cost (typically 1), and we seek the

sequence of operations that minimizes the total cost. Different variants of edit distance may assign different costs to different operations (e.g., Levenshtein distance uses unfirom costs, while other variants may weight substitutions differently)

1.2.3 Example: Alignment of Two Strings

To better understand edit distance, let's examine how two strings can be aligned to show their differences. Consider the string "INTENTION" and "EXECUTION":

```
I  N  T  E  *  N  T  I  O  N
|  |  |  |  |  |  |  |  |  |
*  E  X  E  C  U  T  I  O  N
```

In this alignment:

- The asterisk (*) represents a gap, indicating an insertion or deletion operation
- Vertical bars (|) connect corresponding positions between the two strings
- Characters that match are aligned vertically (E, T, I, O, N)
- Characters that differ indicate a substitution operations

Operations needed to transform "INTENTION" to "EXECUTION"

1. Delete 'I' at position 1
2. Substitute 'N' with 'E' at position 2
3. Substitute 'T' with 'X' at position 3
4. Keep 'E' (match)
5. Insert 'C' at position 5
6. Substitute 'N' with 'U' at position 6
7. Keep 'T' (match)
8. Keep 'I' (match)
9. Keep 'O' (match)
10. Keep 'N' (match)

This gives us a total edit distance of **5 operations** (1 deletion + 3 substitution + 1 insertion)

1.2.4 Cost variants: Standard vs Levenshtein

The total distance depends on how we assign costs to each operation:

Definizione 1.2 (Standard Edit Distance (uniform cost))

- Each operation (insertion, deletion, substitution) costs 1
- Total distance for INTENTION $\rightarrow$ EXECUTION: $\mathbf{5}$
- Calculation: 1 (deletion) + 3 $\times$ 1 (substitutions) + 1 (insertion) = 5

Teorema 1.3 (Levenshtein Distance (weighted substitution))

- Insertion costs 1
- Deletion costs 1
- Substitution costs 2 (considered as a deletion + insertion, hence a "double error")
- Total distance for INTENTION $\rightarrow$ EXECUTION: $\mathbf{8}$
- Calculation: 1 (deletion) + 3 $\times$ 2 (substitution) + 1 (insertion) = 8

Why the difference? In the Levenshtein variant, a substitution is viewed as conceptually equivalent to deleting a character and then inserting a different one, thus costing twice as much. This distinction is important when choosing which distance metric to use for a particular relationships.

1.3 Alignment in Computational Biology

In computation biology, sequence alignment is a fundamental technique for comparing DNA, RNA, or protein sequences. The goal is to identify regions of similarity and understand evolutionary relationships.

1.3.1 The alignment problem

Given a sequence of bases:

```
AGGCTATCACCTGACCTCCAGGCCGATGCCC
```

```
TAGCTATCACGACCGCGGTCGATTTGCCCCGAC
```

An alignment:

```
-AGGCTATCACCTGACCTCCAGGCCGA--TGCCC--
TAG-CTATCAC-GACCGC-GGTCGATTTGCCCCGAC
```

1.3.2 Alignment Objective

Given two sequences, align each letter to a letter or gap

The alignment process involves:

- **Matching**: Aligning identical bases (e.g., A with A, G with G)
- **Mismatches**: Aligning different bases (substitutions)

- **Gaps**: Represented by dashes (-), indicating insertions or deletions (indels)

Key considerations:

- The alignment should maximize the number of matches
- Minimize the number of mismatches and gaps
- Gaps are penalized because schemes can be used: match scores, mismatch penalties, and gap penalites

This alignment problem is directly related to edit distance: finding the optimal alignment is equivalent to finding the minimum edit distance between the two sequences.